

INTERNAL SHARING SESSION

AI Sharing

Scaling Collective Intelligence & Automating
Value

What is this sharing about

- sharing of
 - example of how I have used AI for recent projects
 - my 2 cents of opinions/tips/tricks on AI usage
- the sharing might be a bit disorganized, just imagine I am having a lunch talk with you. Do point out areas where I am wrong.
- main purpose: to get the AI conversation going

Leveraging AI More

important to start leveraging AI more as **AI tools matures** to

Improve



Dev Cycle Speed: Ship features significantly faster.



Operational Efficiency: Improving the way we work, not just what we build.



High-Value, less tedious Work: More time for: design, decision-making, quality improvement initiatives.

As AI tools mature, we can no longer brush it off as something that we use just as a chatbot, coding assistance.

We must use AI as much as possible and increase our AI maturity.

Standardizing Our AI toolbox



Shared Toolbox

shared **AGENTS.md**, skills, hooks, plugins, agentic workflows, MCP setup, etc.



Continuous Improvement

Setting guidelines to iteratively evolve our toolbox based on shared learnings.

Continuous Improvement e.g. AGENT.md

Our shared toolbox should constantly evolve with new guardrails and domain knowledge

Feedback Loop: When Agent make a mistake, we update the AGENT.md with new guardrails.

Collective Memory: Over time, It becomes an expert engineer/domain expert with years of our shared experience.

This applies to any artefact that AI works with: AGENT.md, SKILL.md, knowledge base files, etc.

Some tips & tricks on using agent chats

managing a single agent

- how we use a single agents sets the foundation of of we use more AI for more complex workflows
- so here are some common antipatterns, tips and tricks
- in general it is important we manage context well

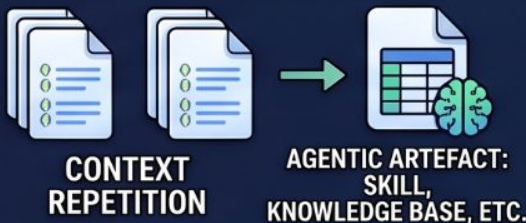
COMMON ANTIPATTERNS OF USING AGENTIC CHATS

MULTIPLE TASKS IN ONE CHAT.



RESOLUTION

IF YOU NEED TO REPEAT CONTEXT
→ CREATE AN ARTEFACT.



"IT SEEMS CORRECT." - PASSIVE ACCEPTANCE

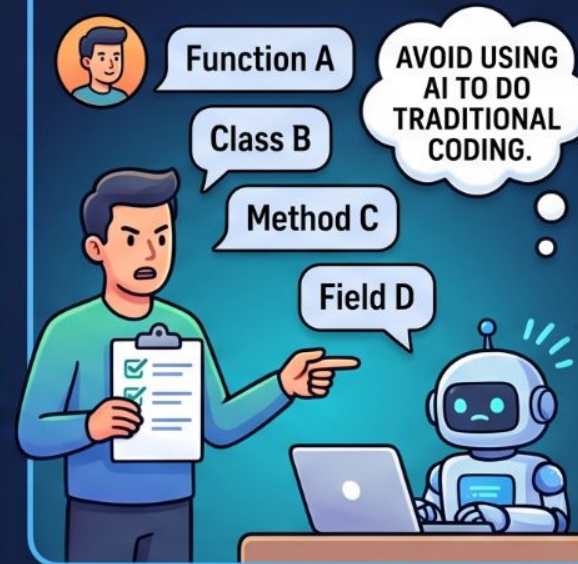


RESOLUTION

GIVE A WAY TO VERIFY

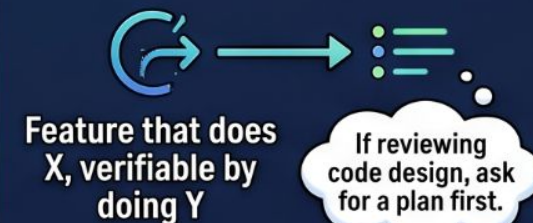


MICRO-MANAGE: TRADITIONAL CODING.

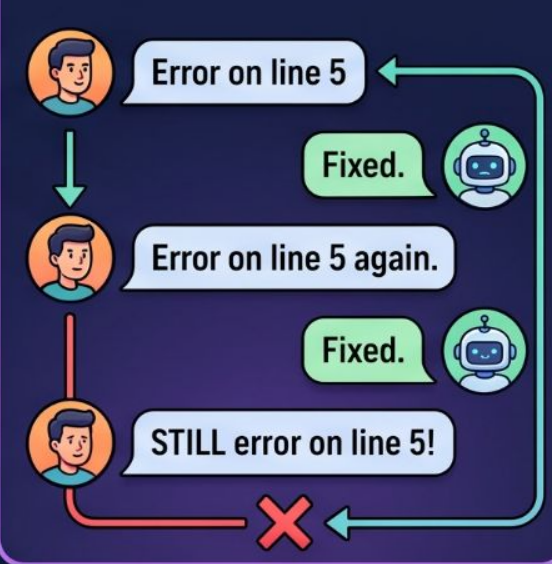


RESOLUTION

ASK FOR VERIFIABLE FEATURE



FAILED CORRECTION LOOP.



RESOLUTION

IF ONE CHAT KEEPS MAKING
SAME MISTAKE...
STOP CORRECTING & RESTART



FURTHER ANTIPATTERNS OF USING AGENTIC CHATS

NOT BEING SPECIFIC



RESOLUTION

RESOLUTION: BE SPECIFIC & PROVIDE CONTEXT

1



Define a Clear Goal:
"improve API response to under 2s"

2



Analyze and Narrow Down:
"bottleneck is in SQL"

3



Propose Targeted Actions:
"consider optimizing SQL query"

NOT USING CLAUDE.MD



RESOLUTION

RESOLUTION: IMPLEMENT A 'CLAUDE.MD' CONTEXT

1



Provide Default Context:
guidelines, guardrails, dos, don'ts

2



Repo-Level Guidelines:
(e.g., general stack, security standards)

3



Module-Level Guidelines:
(e.g., database schema, specific library usages)

more tips & tricks

- be **specific**, be specific, be specific
- provide **reference** whenever you can
 - e.g. write a class/library/test like X
- **constraint**
 - don't do too much in one context window e.g. 1 chat = 1 task
- when debugging: **describe symptoms**, avoid given your biased guesses
- let claude **interview** you
 - "I want to create splitwise clone, interview me for what you need to get started"
 - "I want to create API for payment, interview me for what you need to get started"
- keep context clean to your main task
 - use **subagent** to do research/small task that you think might pollute your context window
 - use **/context** periodically to check your window
 - i. reasoning **degrades** significantly from 40%, 50% onwards (research shows)
- effort level: always use **high/max**
 - better to do it once correctly and twice = save tokens
-

**Agentic tools:
skills, hooks, MCP**

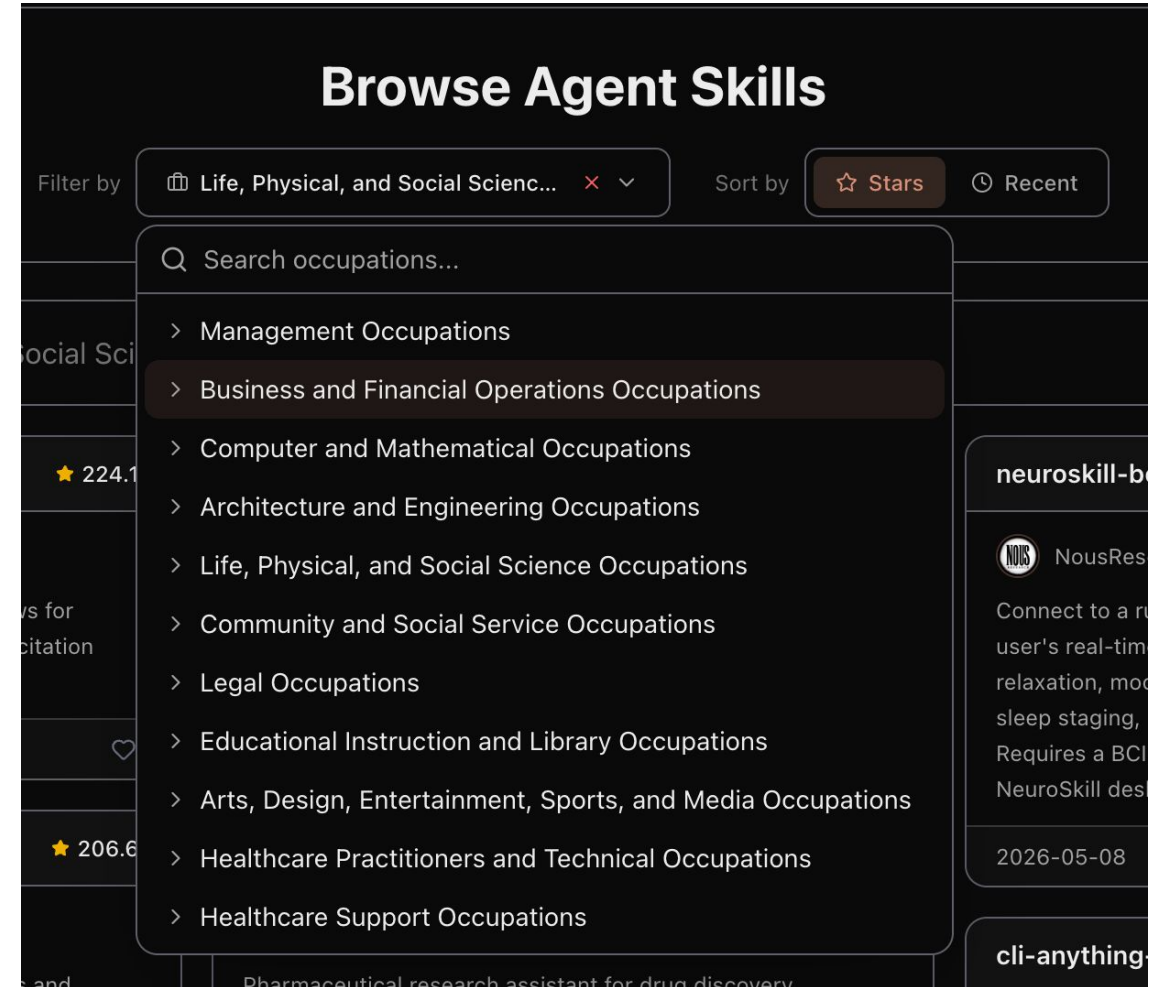
Agentic Skills

- Type of skills
 - Procedures (SOP)
 - i. Claude will execute step by step
 - knowledge, guidelines, rules, persona
 - i. Claude will do something with these guiding information
- modes of invocation
 - explicit invocation /code-review /roast-me
 - model invocation - agent knows your current list of skills, invoke automatically based on your request
 - i. we can disable this for each skill

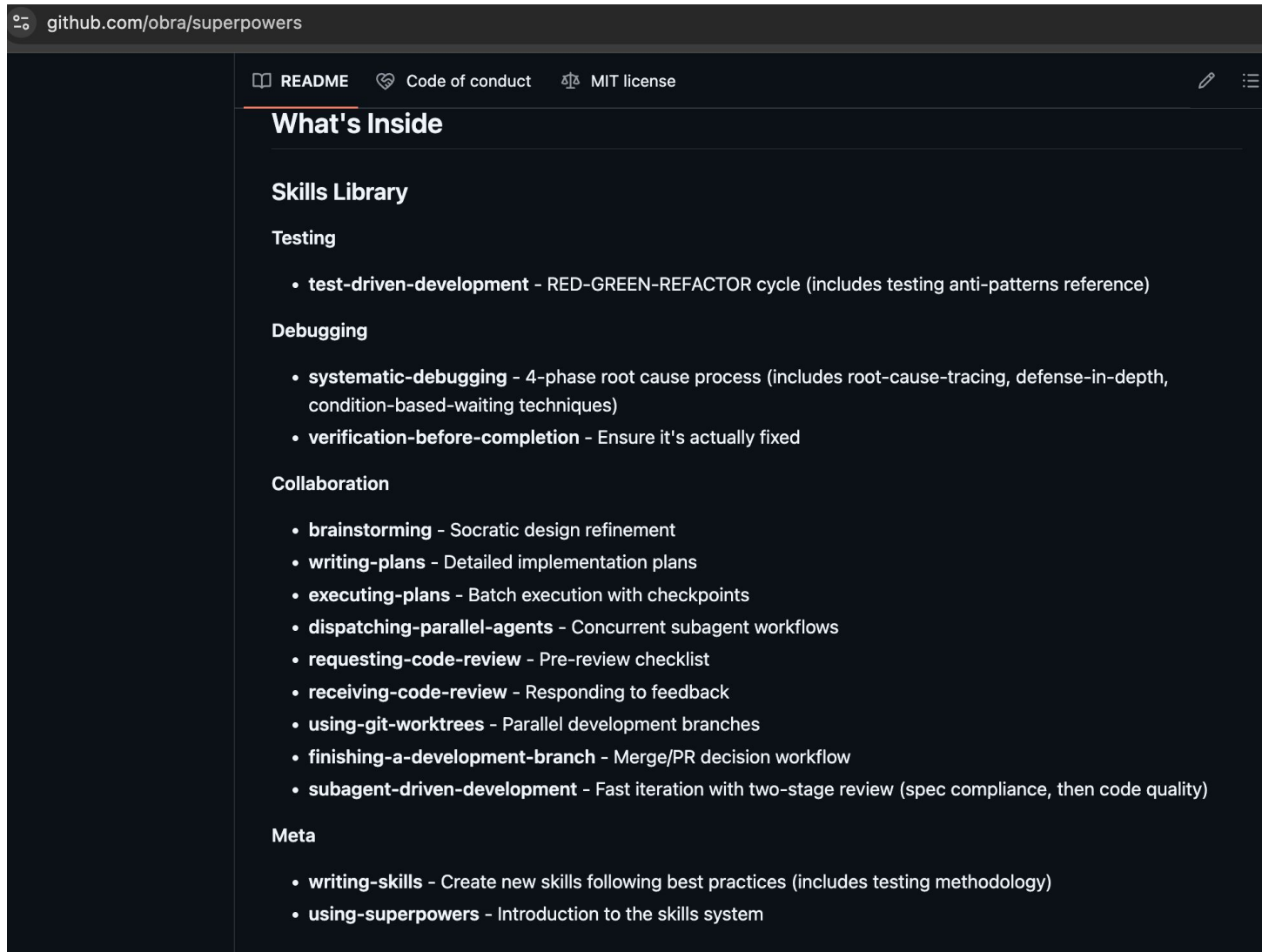
Skills on the marketplace, internet

- There are a ton of useful skills out on the internet
 - for all kind of things not just software development
- should keep up to date with what's available out there
 - as some of the skills are very powerful

<https://skillsmp.com/search>

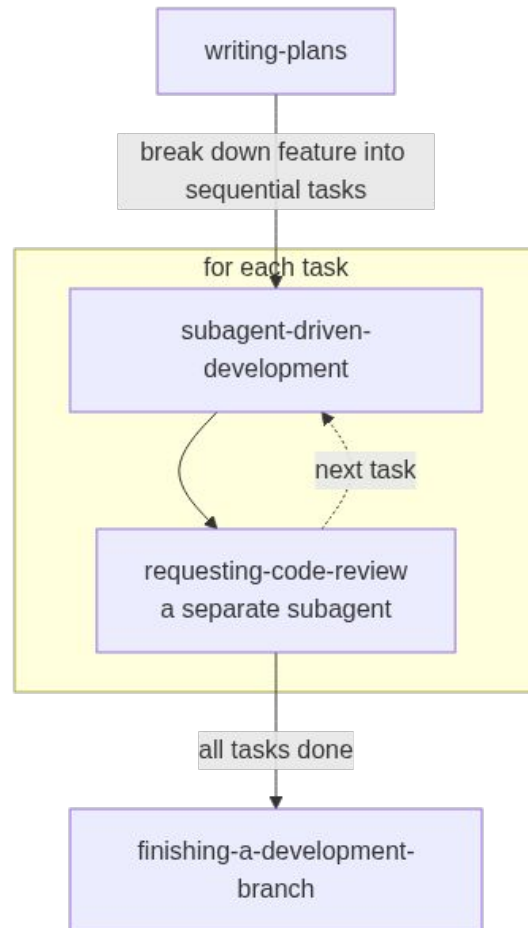


Skills example: superpowers 250k stars on github



Skills example: superpowers

flow



“memory” for the running skills

```
.superpowers / sdd
├── .gitignore
├── progress.md
├── review-85e5f3c..aa16ea8.diff
├── review-696eb4e..adb9df7.diff
├── review-4888545..696eb4e.diff
├── review-a32d6d1..4888545.diff
├── review-a32d6d1..aa16ea8.diff
├── review-adb9df7..85e5f3c.diff
├── task-1-brief.md
├── task-1-report.md
├── task-2-brief.md
├── task-2-report.md
├── task-3-brief.md
├── task-3-report.md
├── task-4-brief.md
├── task-4-report.md
├── task-5-brief.md
├── task-5-report.md
└── .vscode
```

Writing your own skill

- it is very useful to write our own skill too for your specific use case
- actually you don't even need to write your own skill, just use **a /write-a-skill skill to write the skill you want**

skill example: google outbound feed

```
{
  "@context": "http://schema.org",
  "@type": "DataFeed",
  "name": "Viki Google Cast Feed for Home Integrations",
  "dateModified": "2026-06-26T22:04:50+00:00",
  "dataFeedElement": [
    {
      "@context": [
        "http://schema.org",
        {
          "@language": "en"
        }
      ],
      "@id": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30",
      "@type": "TVEpisode",
      "url": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30",
      "name": "Episode 30",
      "partOfSeries": {
        "@type": "TVSeries",
        "@id": "https://www.viki.com/tv/37654c-love-of-thousand-years",
        "name": "Love of Thousand Years"
      },
      "partOfSeason": {
        "@type": "TVSeason",
        "@id": "https://www.viki.com/tv/37654c-love-of-thousand-years-season-1",
        "seasonNumber": 1
      },
      "episodeNumber": 30,
      "description": "Fu Jiu Yun (Zheng Ye Cheng), an immortal deity, pursues a mortal woman across 1,000 years and 10 different lifetimes, per love. When she is reborn as a princess, a neighboring evil prince and his demonic allies destroy her family and realm. Surviving by taking her (Si) identity, the princess seeks revenge with Fu Jiu Yun's help. As they form a deep love bond, they learn their romance is destined to be trag them to make critical choices that will impact their lives and her subjects forever. Based on a novel by Shu Si Lang, this is a 2020 Chinese dr Hui Yu.",
      "image": {
        "@context": "http://schema.org",
        "@type": "ImageObject",
        "contentUrl": "https://1.vikiplatform.com/v/1178714v/7536982506.jpg?x=b&a=0x0",
        "additionalProperty": [
          {
            "@type": "PropertyValue",
            "name": "contentAttributes",
            "value": [
              "iconic",
              "poster"
            ]
          }
        ]
      },
      "inLanguage": "zh-sg",
      "potentialAction": [
        {
          "@type": "WatchAction",
          "target": [
            {
              "@type": "EntryPoint",
              "urlTemplate": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30?utm_source=global_search_or_media_action&u",
              "inLanguage": "zh-sg",
              "actionPlatform": "viki"
            }
          ]
        }
      ]
    }
  ]
}
```

skill example: google outbound feed

```
{
  "@context": "http://schema.org",
  "@type": "DataFeed",
  "name": "Viki Google Cast Feed for Home Integrations",
  "dateModified": "2026-06-26T22:04:50+00:00",
  "dataFeedElement": [
    {
      "@context": [
        "http://schema.org",
        {
          "@language": "en"
        }
      ],
      "@id": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30",
      "@type": "TVEpisode",
      "url": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30",
      "name": "Episode 30",
      "partOfSeries": {
        "@type": "TVSeries",
        "@id": "https://www.viki.com/tv/37654c-love-of-thousand-years",
        "name": "Love of Thousand Years"
      },
      "partOfSeason": {
        "@type": "TVSeason",
        "@id": "https://www.viki.com/tv/37654c-love-of-thousand-years-season-1",
        "seasonNumber": 1
      },
      "episodeNumber": 30,
      "description": "Fu Jiu Yun (Zheng Ye Cheng), an immortal deity, pursues a mortal woman across 1,000 years and 10 different lifetimes, per love. When she is reborn as a princess, a neighboring evil prince and his demonic allies destroy her family and realm. Surviving by taking her (Si) identity, the princess seeks revenge with Fu Jiu Yun's help. As they form a deep love bond, they learn their romance is destined to be trag them to make critical choices that will impact their lives and her subjects forever. Based on a novel by Shu Si Lang, this is a 2020 Chinese dr Hui Yu.",
      "image": {
        "@context": "http://schema.org",
        "@type": "ImageObject",
        "contentUrl": "https://1.vikiplatform.com/v/1178714v/7536982506.jpg?x=b&a=0x0",
        "additionalProperty": [
          {
            "@type": "PropertyValue",
            "name": "contentAttributes",
            "value": [
              "iconic",
              "poster"
            ]
          }
        ]
      },
      "inLanguage": "zh-sg",
      "potentialAction": [
        {
          "@type": "WatchAction",
          "target": [
            {
              "@type": "EntryPoint",
              "urlTemplate": "https://www.viki.com/videos/1178714v-love-of-thousand-years-episode-30?utm_source=global_search_or_media_action&u",
              "inLanguage": "zh-sg",
              "actionPlatform": "viki"
            }
          ]
        }
      ]
    }
  ]
}
```

Content markup properties of TV shows

◆ Page Summary

This section provides the details of the content markup properties for the `TVSeries`, `TVEpisode`, and `TVSeason` entity types.

Providing `TVSeason` as a full entity is optional. See the [TVSeries, TVSeason, and TVEpisode relations](#) section for details.

Specification table

TVSeries TVEpisode TVSeason		
Property	Expected Type	Description
@context	Text 	Required - Always set to <code>["http://schema.org", {"@language": "xx"}]</code> Where "xx" represents the language of the strings in the feed. Each root entity context should have its @language set to the appropriate language code in BCP 47 format  format. For example, if the language is set to Spanish, the names are assumed to be in Spanish even if the subtitle/dub language is in English.
@type	Text 	Required - Always set to <code>TVSeries</code>.
@id	URL 	Required - The content's identifier in URI format; for example, <code>https://example.com/1234abc</code>. @id must meet the following requirements: - Globally unique across your catalog - The ID should be stable and not change over time (even if the url property of the show changes). It will be treated as an opaque string and does not have to be a working link. - In the form of Unified Resource Identifier (URI) - The domain used for the @id value must be owned by your organization.

- 1 list for each of
- TV Episode
 - TV Season
 - TV Series
 - Movie

Writing your own skill

Dear Claude:

help me write a skill:

- ...some context about google feed
- ...context about our code (generator class, usage, etc)
- instructions (entrypoint in the sense of a Docker entrypoint): the skill should ask for a bug report and fix the generator class to generate the the
 - list specific dos & don'ts: must ensure test pass
- embed these doc links for agents to use when needed:

TVSeries

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvseries>

TVEpisode

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvepisode>

TVSeason

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvseason>

Movie

Reference: <https://developers.google.com/actions/media/reference/data-specification/movies-specification>

Heck,

if you don't know how to best write a prompt for make a skill
use a **make-a-prompt-to-make-a-skill skill** to **grill you** for the
questions needed to generate a good make-a-skill prompt

or end with

“ask me for anything else that could be needed”

name	google-cast-feed-bugfix
description	Use when given a bug report (critical or quality improvement) against the Viki Google Cast feed generator — field values wrong, missing required fields, malformed shapes, spec violations, or schema.org compliance issues in TVSeries, TVEpisode, TVSeason, or Movie output.

Google Cast Feed Bugfix

Overview

Fix bugs in the Viki Google Cast feed generator. The generator produces a JSON-LD DataFeed of schema.org entities (TVSeries, TVEpisode, TVSeason, Movie) for Google Home / Chromecast discovery. Bugs are typically spec violations — wrong field values, missing required fields, wrong formats, or structural shape errors.

Key Files

File	Role
app/services/outbound_feed/google_cast_page.rb	Feed generator — one page of entities. All render_* methods live here.
app/services/outbound_feed/google_cast.rb	Orchestrator — calls <code>GoogleCastPage.new(...).generate_content</code> per page
spec/services/outbound_feed/google_cast_spec.rb	Spec — golden snapshot + unit tests
spec/fixtures/google_cast/sample_feed.json	Golden snapshot fixture — update when expected output changes
spec/fixtures/google_cast/containers.json	Channel/container fixture data
spec/fixtures/google_cast/videos.json	Episode/movie video fixture data

A "channel" or "container" maps to `Oceanus::Container` — it is either a TV series (`type: 'series'`) or a movie (`type: 'film'`).

Workflow

1. Understand the bug report
 2. Look up the spec for the affected entity type
 3. Find the render_* method in google_cast_page.rb
 4. Fix the implementation
 5. Update sample_feed.json if the expected output changes
 6. Update or add spec examples as needed
 7. Run the spec to verify

Step 2 — Look Up the Spec

Consult the authoritative spec for the entity. Use WebFetch if you need to recheck field constraints.

TVSeries

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvseries>

TVEpisode

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvepisode>

TVSeason

Reference: <https://developers.google.com/actions/media/reference/data-specification/tv-shows-specification#tvseason>

Movie

Reference: <https://developers.google.com/actions/media/reference/data-specification/movies-specification>

Feed overview / WatchAction / ActionAccessSpecification

Reference: <https://developers.google.com/actions/media/reference>

using the skill



Improve entity quality

Issue 	↓ Entities	Trend
Property "episodeNumber" exceeds maximum value	8,432	
Disallowed word in "name"	184	
Invalid value in field "availabilityEnds" (in "potentialAction.actionAccessibilityRequirement")	118	
Invalid character in "name" (in "partOfSeries")	28	
Episode title is the same as the series title	18	

/google-cast-feed-bugfix

help me fix this bug for TV episode:

"Property "episodeNumber" exceeds maximum value"

example:

"""

"partOfSeason": {

 "@type": "TVSeason",

 "@id": "https://www.viki.com/tv/36478c-ask-us-anything-season-1",

 "seasonNumber": "1"

},

 "episodeNumber": "201",

 "description": "\"Ask Us Anything\" is a popular Korean variety show featuring a dynamic panel of comedians and celebrities who tackle life's challenging questions with humor and wisdom. The star-

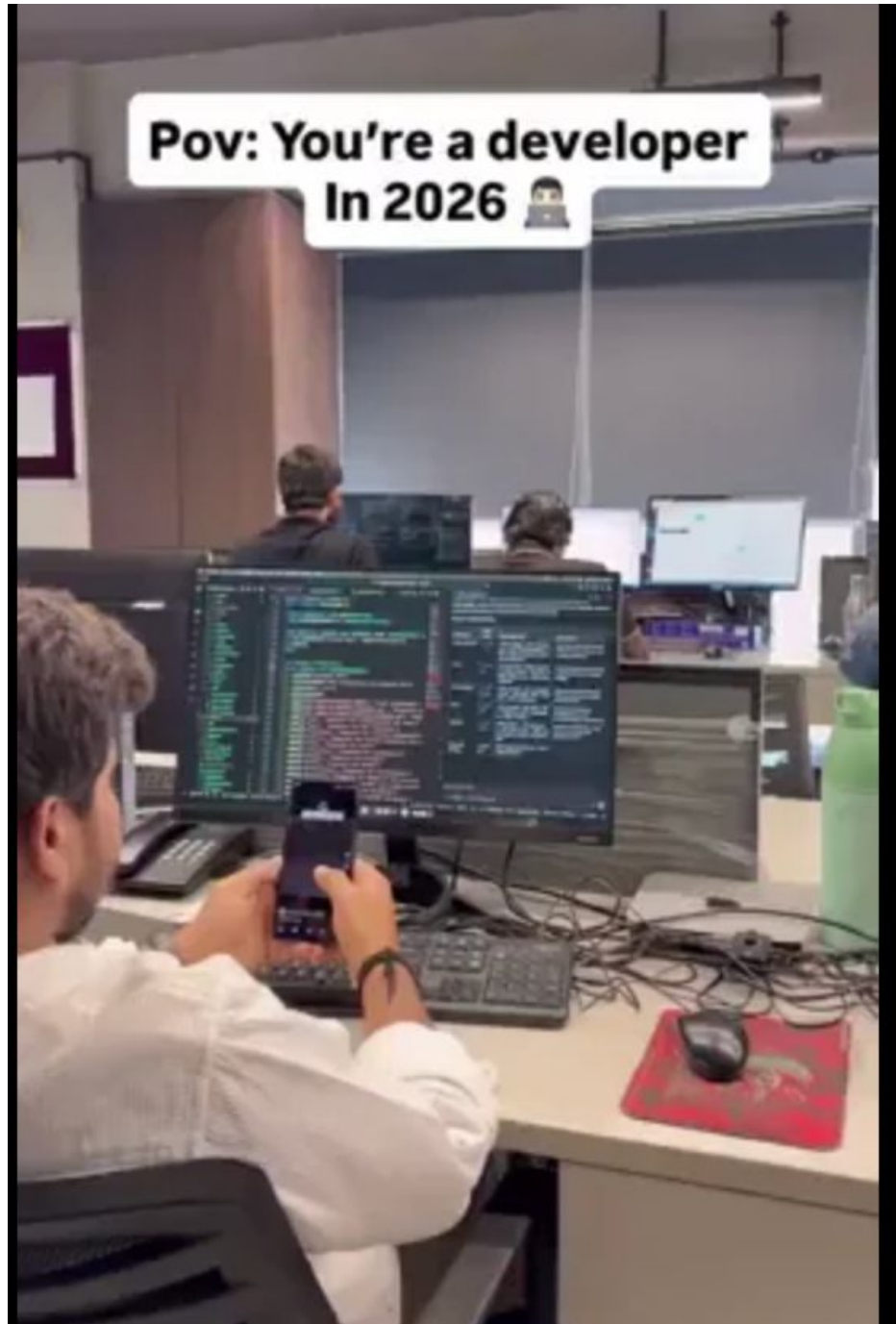
},

 "episodeNumber": "201",

 "description": "\"Ask Us Anything\" is a popular Korean variety show featuring a dynamic panel of comedians and celebrities who tackle life's challenging questions with humor and wisdom. The star-studded lineup includes Kang Ho Dong, Seo Jang Hoon, Kim Young Chul, Lee Soo Geun, Kim Hee Chul, Min Kyung Hoon, and Lee S",

"""

Pov: You're a developer
In 2026 🧑💻



hooks & MCP.

- honestly haven't used much, looking to use it more

hooks & MCP

- **hooks**
 - key distinction: hooks are **deterministic**
 - whereas LLM is non-deterministic

classic example

- auto approve read file operations
- run test/lint after end of tool use

Hook	Trigger Timing	Typical Use Cases
PreToolUse	Before Claude calls a tool	Intercept dangerous operations
PostToolUse	After Claude calls a tool	Auto-formatting, auto-testing
PermissionRequest	When user authorization is required	Auto-approve low-risk operations
Stop	When Claude completes a turn	Push to continue execution
PostCompact	After context compression	Inject key instructions to prevent forgetting
PermissionDenied	After Auto mode classifier denies an operation	Log denied operations, notify user, trigger alternatives

hooks & MCP

- **MCP**
 - way to interface with external system and their data without going through traditional API calling, in an more “LLM” way
 - underneath the hood still API call happens but the protocol is natural language
- classic use cases
 - slack summarizer/info gatherer
 - JIRA summary/updates
 - datadog debugging/ insights

Advanced Usage

Exploring Agentic Workflows & Pipelines

Agentic Workflows/pipelines

- there is no one concrete definitions in the industry, for anthropic it means one thing, for openai it might mean another, etc.
- ultimately, just means defining a reusable agentic pipeline/sequence/graph of work
- we can define dependencies, specify what should be done in each unit of work
 - unit of work: calling agents to execute something, research sth, invoke skills, APIs, scripts, etc. anything tbh
- patterns
 - fan-out, fan-in (parallelization, deploying multiple agents and collate results)
 - adversary review (separate agents for non-biased review)
 - control flow (routing to different agents depending on input)
 - feedback loop (keep running new agents until stop-loop condition is satisfied)
 - types of feedback: running tests, calling API, visual
- it can be as simple as chaining together a series of skill invocations, to more complicated graph of work
- Need to explore this further as this is potentially very powerful

```
1  name: PRD to PR
2  description: Full pipeline from Confluence PRD to merged GitHub PRs
3  version: "1.1"
4
5  stages:
6 > - id: prd-read ...
48
49 > - id: technical-foundation ...
92
93 > - id: review-docs ...
128
129 > - id: publish-docs ...
160
161 > - id: create-tickets ...
194
195 > - id: implement-ticket ...
228
229 > - id: self-review ...
252
253 > - id: ship-ticket ...
279
```



```

1  schema_version: 1
2  id: swing-opportunity-daily
3  display_name: Swing Opportunity Daily
4  cadence: daily
5  estimated_minutes: 40
6  target_users:
7    - part-time-swing-trader
8  difficulty: intermediate
9  api_profile: fmp-required
10
11  when_to_run: >-
12    Only after market-regime-daily has produced a non-restrictive exposure
13    decision. Identifies swing trade candidates and builds entry plans.
14  when_not_to_run: >-
15    Do not run when the latest market-regime-daily exposure_decision is
16    cash-priority or restrictive. Do not use as a standalone screener without
17    the regime gate.
18
19  required_skills:
20    - vcp-screener
21    - drawdown-circuit-breaker
22    - technical-analyst
23    - position-sizer
24    - trader-memory-core
25    - pre-trade-discipline-gate
26  optional_skills:
27    - stockbee-momentum-burst-screener
28    - stockbee-exhaustion-hammer-screener
29    - canslim-screener
30    - breakout-trade-planner
31    - theme-detector
32

```

```

30
31  steps:
32    - step: 1
33      name: Check account circuit breaker
34      skill: drawdown-circuit-breaker
35    > produces: ...
36      decision_gate: true
37    > decision_question: >-...
38
39    - step: 2
40      name: Run VCP screener
41      skill: vcp-screener
42    > produces: ...
43      decision_gate: false
44
45    - step: 3
46      name: Run Stockbee momentum burst screener
47      skill: stockbee-momentum-burst-screener
48      optional: true
49    > produces: ...
50      decision_gate: false
51
52    - step: 4
53      name: Run Stockbee exhaustion hammer screener...
54      decision_gate: false
55
56    - step: 5
57      name: Run CANSLIM screener...
58      decision_gate: false
59
60    - step: 6
61      name: Theme detection cross-check
62      skill: theme-detector
63    > optional: true...
64      decision_gate: false
65
66    - step: 7...

```

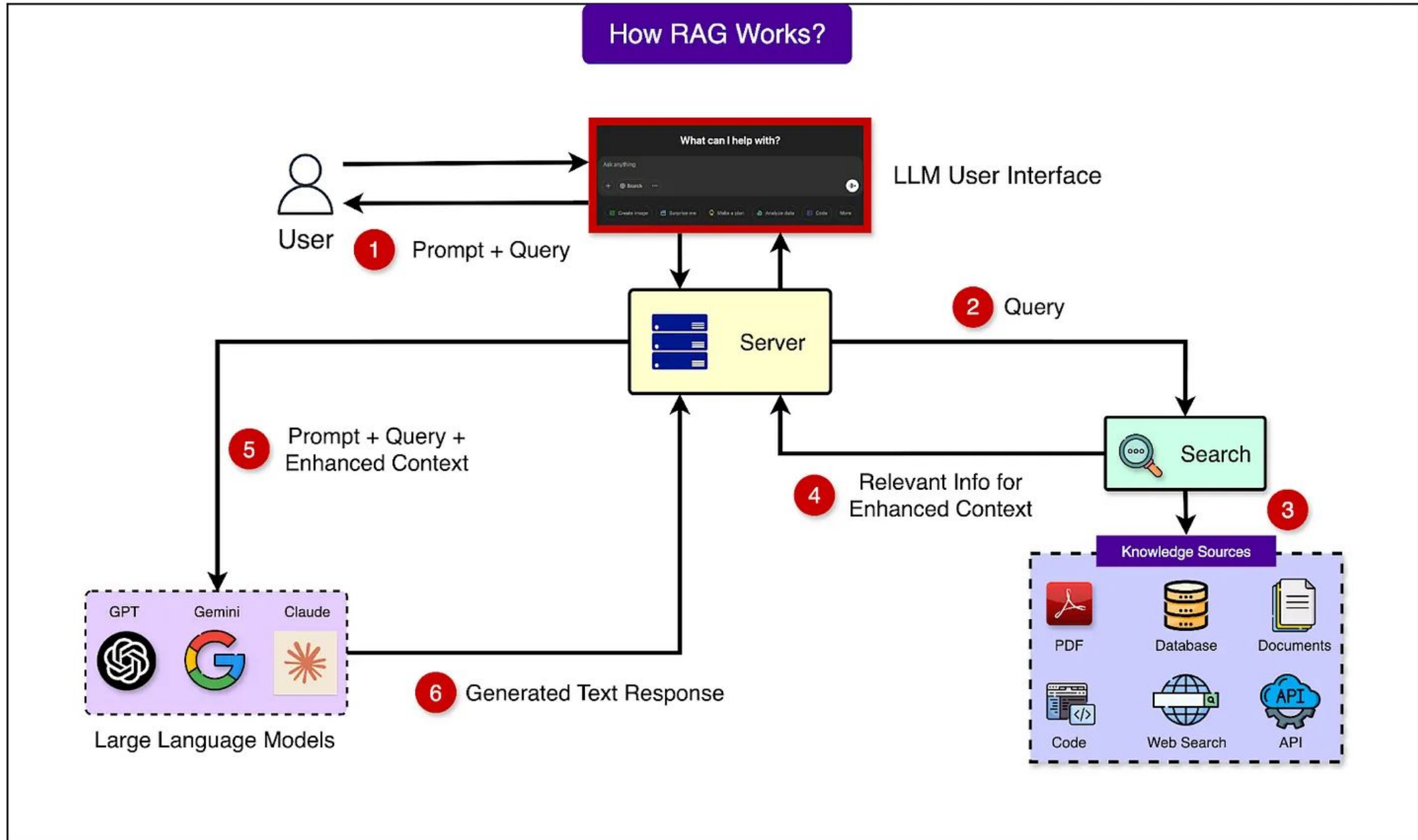
RAG



RAG (Retrieval Augmented Generation)

- actually i thought I was not using RAG.
- but I am using it frequently by asking AI to fetch certain websites
- which begs the question: what is RAG?

RAG - Retrieving what?, Augmenting what?



RAG

- web is public data
- when you want to ask LLM to leverage your own private/organizational data, you want to store these in your own database
- this is where you want to set up a vector database
- key challenges/technologies to deal with
 - data ingestion
 - embedding model
 - vector database
 - orchestration
 - maintenance & fine-tuning the data for best result

- I have probably missed out on some areas we should be looking into
 - multi-agent systems (moving past the "single AI assistant" era)
 - spec driven development
- The AI landscape is fast-moving and who knows what new technologies might appear
- but we should keep learning and adapt to these new tools, while not forgetting to maintain our engineering fundamentals